
Project report on shortest path portfolios

Finn Harbeke

supervised by Miltiadis Stouras and Ola Svensson

June 9, 2026

1 Introduction

Think of a navigation service that suggests a route through the city. The travel time along each road changes throughout the day—congestion builds during rush hour, eases overnight, and varies from one day to the next. The service has access to historical speed data and can estimate what the roads will look like on average, but when the user actually drives, the realised travel times are a sample from that distribution. Which route should the service recommend?

A typical approach is to compute the shortest path with respect to the current (known) travel times, for instance by running Dijkstra’s algorithm at query time. This returns the optimal path for the current conditions, every time, but it is expensive: each query costs $O(m \log n)$ time, where m is the number of edges and n the number of vertices in the graph. If the navigation service handles many queries per second the computational load adds up. You would think that much of this computation is repetitive, even if slightly different every time. In principle the entire costs could be shifted to a preprocessing phase by computing and storing an answer for every possible traffic scenario, but the space required grows with the number of scenarios and quickly becomes impractical.

The problem of answering many shortest-path queries over a distribution of edge costs we refer to as the *stochastic shortest path problem* (problem 1). A recent line of work [Dry+24] defines the *portfolio optimization* problem which our problem can be considered an instance of; during preprocessing we select a small set of candidate paths (a portfolio) for each source–target pair, so that in any traffic scenario at least one of them is cheap. At query time we only need to evaluate the portfolio under the current conditions and return the cheapest—an operation that is linear-time in the size of the portfolio and the size of its paths. The tradeoff between the size of the portfolio and the expected approximation quality is thus the central

question.

This project explores this tradeoff on a real-world traffic dataset. We take the road network of a Shanghai district with travel speeds aggregated over 10-minute intervals throughout April 2015, which gives us a concrete distribution of edge cost functions. For this instance we implement three portfolio algorithms: a greedy algorithm that incrementally builds a portfolio by minimising expected cost over the full path set, a frequency-based selection that picks the paths that are shortest the most often, and a lightweight penalty scheme that repeatedly runs Dijkstra on expected edge costs while discouraging reuse of edges. We evaluate these algorithms in terms of the expected approximation factor they achieve for different portfolio sizes and the preprocessing and query times they require, and we compare them against the baseline of running Dijkstra at every query.

Our goal is not to claim that one algorithm is universally best, but rather to understand the practical behaviour of the portfolio model on a realistic input. We find that on the Shanghai dataset, portfolios of modest size ($k \approx 8$) achieve expected costs within hundredths of a percent of the optimal, and that the frequency-based approach—simple and fast—performs nearly as well as the greedy algorithm which has to iterate over the exponentially many candidate paths. The penalty-based scheme, while the cheapest to precompute, converges more slowly and illustrates the cost of ignoring the distributional information beyond the mean.

2 Preliminaries

Let $G = (V, E)$ be a directed graph with $n = |V|$ vertices and $m = |E|$ edges. We consider a stochastic setting in which the edge costs are random. Let $\mathcal{C}$ be a set of cost functions from E to $\mathbb{R}^{\geq 0}$ and $\mathcal{D}$ a distribution over them. So, $c \sim \mathcal{D}$

is a realised cost function assigning non-negative real cost $c(e)$ to every edge in G . A path is a sequence of vertices $P = (v_0, v_1, \dots, v_p) \in V \times V \times \dots \times V$, such that $e_i = (v_{i-1}, v_i) \in E$ for every $1 \leq i \leq p$. For paths' cost we write $f_c(P)$ as the sum of the costs of its edges. That is, $f_c(P) := \sum_{i=1}^p c(e_i)$ for $c \in \mathcal{C}$.

For two vertices $s, t \in V$, the *shortest path* based on the random cost function $c \sim \mathcal{D}$ is

$$P_{s,t}^* = \arg \min_{P \in \mathcal{P}_{s,t}} f_c(P)$$

where $\mathcal{P}_{s,t}$ is the set of all directed paths from s to t in G and we break ties according to a fixed deterministic rule. Since the cost function follows the distribution $\mathcal{D}$, $P_{s,t}^*$ is a categorical random variable taking values in $\mathcal{P}_{s,t}$.

In the same manner, we define a random variable that denotes the cost of the shortest path, i.e. the distance

$$d_{s,t} = \min_{P \in \mathcal{P}_{s,t}} f_c(P)$$

which takes non-negative, real values.

2.1 Query Model

We study the following computational problem.

Problem 1 (Stochastic Shortest Path Problem). Given the tuple $(G, \mathcal{C}, \mathcal{D})$ of graph and the distribution over edge cost functions, preprocess the available information to answer queries of the following form:

Query: $(s, t) \in V \times V$, with realised $c \sim \mathcal{D}$.

Answer: Directed path P from s to t that is as cheap as possible under f_c .

Let ALG be an algorithm solving problem 1, then the quality of ALG's query answers can be measured in different manners. For a fixed source-target pair (s, t) on a fixed input $(G, \mathcal{C}, \mathcal{D})$ we define ALG's expected approximation factor:

$$\alpha(\text{ALG}, (G, \mathcal{C}, \mathcal{D}); s, t) = \frac{\mathbb{E}_{c \sim \mathcal{D}} [f_c(P_{\text{ALG}})]}{\mathbb{E}_{c \sim \mathcal{D}} [f_c(P_{s,t}^*)]}$$

When quantifying ALG's answer quality on G as a whole, we can use different aggregating metrics over all (s, t) -pairs. Of natural interest is an upper bound on the expected approximation factors of all (s, t) -pairs. Due to limitations in compute resources we consider the observed mean

$$\bar{\alpha}_r(\text{ALG}, (G, \mathcal{C}, \mathcal{D})) = \frac{1}{r} \sum_{(s,t) \sim \mathcal{U}\{ST(G)\}} \frac{\mathbb{E}_{c \sim \mathcal{D}} [f_c(P_{\text{ALG}})]}{\mathbb{E}_{c \sim \mathcal{D}} [f_c(P_{s,t}^*)]},$$

where $\mathcal{U}\{A\}$ is the discrete uniform distribution over the set A and $ST(G) = \{(s, t) \in V \times V \mid t \text{ reachable from } s \text{ in } G\}$. As a quality measure, $\bar{\alpha}_r(\text{ALG}, (G, \mathcal{C}, \mathcal{D}))$ serves as an unbiased estimator of the expected $\alpha(\text{ALG}, (G, \mathcal{C}, \mathcal{D}); s, t)$ for $(s, t) \sim$

$\mathcal{U}\{ST(G)\}$. It captures how much more ALG's answer is going to cost for a uniformly at random query (s, t) and a realised edge cost function c in comparison to the optimal solution for that query, $f_c(P_{s,t}^*)$.

Next to the above quality measure, we define three key metrics that matter for the analysis of any given ALG solving problem 1. Let $T^0(\text{ALG})$ be the worst-case preprocessing time, dependent on $n = |V|, m = |E|$ and N , that is the number of sampled c 's taken from $\mathcal{D}$ in order to learn the distribution. The corresponding worst-case query time $T^q(\text{ALG})$ denotes the worst-case time it takes ALG to provide an answer for any query (s, t, c) . Lastly, $S(\text{ALG})$ denotes the space required by ALG to store the datastructure computed in preprocessing.

A naive approach could be to precompute a single path $P_{s,t}$ for every pair (s, t) ; this uses $O(n^3)$ storage and answers queries instantly, but the returned path may be far from optimal in many events. Still, one can take $\mathcal{C}$ and $\mathcal{D}$ into consideration and make the best out of constant worst-case query time. The even more trivial solution is to compute a shortest path from scratch per query; this requires no preprocessing and is expensive at query time, but it always returns the optimum answer. In fact, this is a way to reduce the stochastic setting to the traditional shortest path problem.

Algorithm 1 Stateless Dijkstra Algorithm, solving [Stochastic Shortest Path Problem](#) optimally.

```

1: procedure PREPROCESS( $G, \mathcal{C}, \mathcal{D}$ )
2:   data  $\leftarrow \emptyset$ 
3:   return data

4: procedure QUERY( $s, t, c, \text{data}$ )
5:    $P \leftarrow \text{DIJKSTRA}(G, s, t, c)$ 
6:   return  $P$ 
```

The main characteristic that we want to improve on in this project is the fact that $T^q(\text{Algorithm 1}) = O(n \log n)$. For completeness sake let us explicitly state that both $T^0(\text{Algorithm 1})$ both $S(\text{Algorithm 1})$ is zero and $\bar{\alpha}_r(\text{Algorithm 1}, (G, \mathcal{C}, \mathcal{D})) = 1$ for any $(G, \mathcal{C}, \mathcal{D})$.

2.2 Portfolio Method

The problem we study can be framed as an instance of the *Portfolio Optimization* framework introduced by [Dry+24]. In the general formulation, a portfolio problem is described by a tuple $(I, \mathcal{V}, \mathcal{D}, k)$ where I is the set of feasible solutions to a combinatorial problem Π , $\mathcal{V}$ is a set of value functions over I , $\mathcal{D}$ is a distribution over $\mathcal{V}$, and k is the desired portfolio size. The goal is to select k solutions $S_1, \dots, S_k \in I$ offline, using only knowledge of $\mathcal{D}$, to optimise the expected value of the best among them:

$$\text{value}(\{S_1, \dots, S_k\}) = \mathbb{E}_{v \sim \mathcal{D}} \left[\max_{i \in [k]} v(S_i) \right].$$

For our shortest path setting, we work with the minimisation variant. The solution set $I = \mathcal{P}_{s,t}$ is the set of all directed s - t paths. Instead of value functions, we use cost functions. And importantly, the cost functions in $\mathcal{C}$ are not given directly over paths, but over edges instead. But for some $c \in \mathcal{C}$ we defined f_c as the cost function for paths. Let $\mathcal{F}$ thus be the set of all path-cost functions. Then we can write $\mathcal{D}_{\mathcal{F}}$ as the distribution over $\mathcal{F}$ that is obtained by taking f_c for $c \sim \mathcal{D}$. Given these notations, we can define the portfolio problem.

Let $(G, \mathcal{C}, \mathcal{D})$ be an instance of problem 1 with a single fixed query (s, t) , but the realisation of c unknown in advance.

Problem 2 ((s, t) -Portfolio). Given the tuple $(\mathcal{P}_{s,t}, \mathcal{F}, \mathcal{D}_{\mathcal{F}}, k, s, t)$ find a collection of k candidate paths $\mathcal{K}_{s,t} = \{P_1, \dots, P_k\}$, called a *portfolio*, such that

$$\text{cost}(\mathcal{K}_{s,t}) = \mathbb{E}_{f_c \sim \mathcal{D}_{\mathcal{F}}} \left[\min_{i \in [k]} f_c(P_i) \right]$$

is as small as possible. In particular, we compare its cost with that of the shortest path:

$$\alpha(\mathcal{K}_{s,t}) = \frac{\mathbb{E}_{f_c \sim \mathcal{D}_{\mathcal{F}}} [\min_{i \in [k]} f_c(P_i)]}{\mathbb{E}_{f_c \sim \mathcal{D}_{\mathcal{F}}} [f_c(P_{s,t}^*)]},$$

(s, t) -Portfolio provides us with a straightforward solution strategy for **Stochastic Shortest Path Problem**. Given $O(kn^3)$ space, we can find portfolios for every pair (s, t) of reachable vertices in G , each taking $O(kn)$ space. Then, for any query (s, t, c) we evaluate $\mathcal{K}_{s,t}$ under f_c and return the cheapest path among them.

Note that $\mathcal{P}_{s,t}$ can have a cardinality that is exponential in n . So, if we wanted to use an algorithm that solves problem 2 in order to find answers for problem 1, then we had exponential worst-case runtime no matter what. Instead, we define a sibling problem to problem 2, whose input is polynomial in n, m, N . Instead of the explicit $\mathcal{P}_{s,t}$ and the functions over them we pass G and $\mathcal{C}$ as input and let the algorithm decide if it needs to find all paths between s and t or if it does not actually need the entire set in order to find a fitting subset. The relative cost $\alpha(\mathcal{K}_{s,t})$ can also be computed without being given the explicit $\mathcal{F}$, by just taking $c \in \mathcal{C}$ and computing the path costs as sums of the costs of their edges.

Problem 3 (implicit (s, t) -Portfolio). Given the tuple $(G, \mathcal{C}, \mathcal{D}, k, s, t)$ find a collection of k candidate paths $\mathcal{K}_{s,t} = \{P_1, \dots, P_k\} \subseteq \mathcal{P}_{s,t}$, called a *portfolio*, such that $\alpha(\mathcal{K}_{s,t})$ is as small as possible.

Given this problem, we can now use a black-box algorithm PORTFOLIOALG for it and describe the strategy with which we solve problem 1 in algorithm 2. If we take $k = O(n)$, then all the inputs to PORTFOLIOALG are polynomial in n, m, N and if PORTFOLIOALG's worst-case runtime is polynomial in n, m, N, k , then algorithm 2 has a worst-case runtime that is polynomial in its input length.

Algorithm 2 Portfolio-based algorithm for problem 1 using a black-box PORTFOLIOALG for problem 3.

```

1: procedure PREPROCESS( $G, \mathcal{C}, \mathcal{D}$ )
2:    $k \leftarrow$  chosen parameter
3:   for  $(s, t) \in ST(G)$  do
4:      $\mathcal{K}_{s,t} \leftarrow \text{PORTFOLIOALG}(G, \mathcal{C}, \mathcal{D}, k)$ 
5:   return  $\mathcal{K} = \{\mathcal{K}_{s,t} \mid (s, t) \in ST(G)\}$ 

6: procedure QUERY( $s, t, c, \mathcal{K}$ )
7:    $P \leftarrow \arg \min_{P' \in \mathcal{K}_{s,t}} f_c(P')$ 
8:   return  $P$ 

```

3 Datasets

Throughout sections 4 to 6 we will study the **Stochastic Shortest Path Problem** through the lens of a specific instance. We use a real-world traffic dataset, derived from taxi trajectory data collected by the Shanghai Transportation Information Center (TIC). The data is originally introduced by Wang et al., using the TIC taxi trajectory data of the entire city. In the same year, Wang et al. selected a subgraph thereof, for which there exists an entire time series, for use in metropolitan traffic prediction and made that dataset publicly accessible¹. This subgraph spans between the south-west of Jing'An district and the eastern end of Putuo. The graph does not span a very large area, spanning not more than 3 kilometers in both the north-south as well as the east-west directions. It is depicted in section 3.

3.1 Preprocessing

The raw data consists of average vehicle speeds (in km/h) for each road segment, aggregated over 10-minute intervals from taxi GPS trajectories. For our purposes, we convert these speeds into travel times. Because the coordinates of every intersection is part of the data, we can compute the length of each road segment by using the haversine distance between intersections. This introduces a small source of error for road segments with bends, which seems negligible when looking at section 3 more closely. Thus, for a road segment e of length ℓ_e (in metres) with observed speed s_e (in km/h) at a given time interval, the travel time is

$$c(e) = \frac{\ell_e}{s_e} \cdot \frac{60}{1000}.$$

in minutes.

There are a total of $N = 4320$ 10-minute intervals where the taxi travel speeds have been recorded. This provides us with exactly N edge cost functions. In order to get a feel for the typical speeds and the behaviour patterns of the dataset, we visualize the raw data in section 3.1.

¹<https://github.com/xxArbiter/grnn>

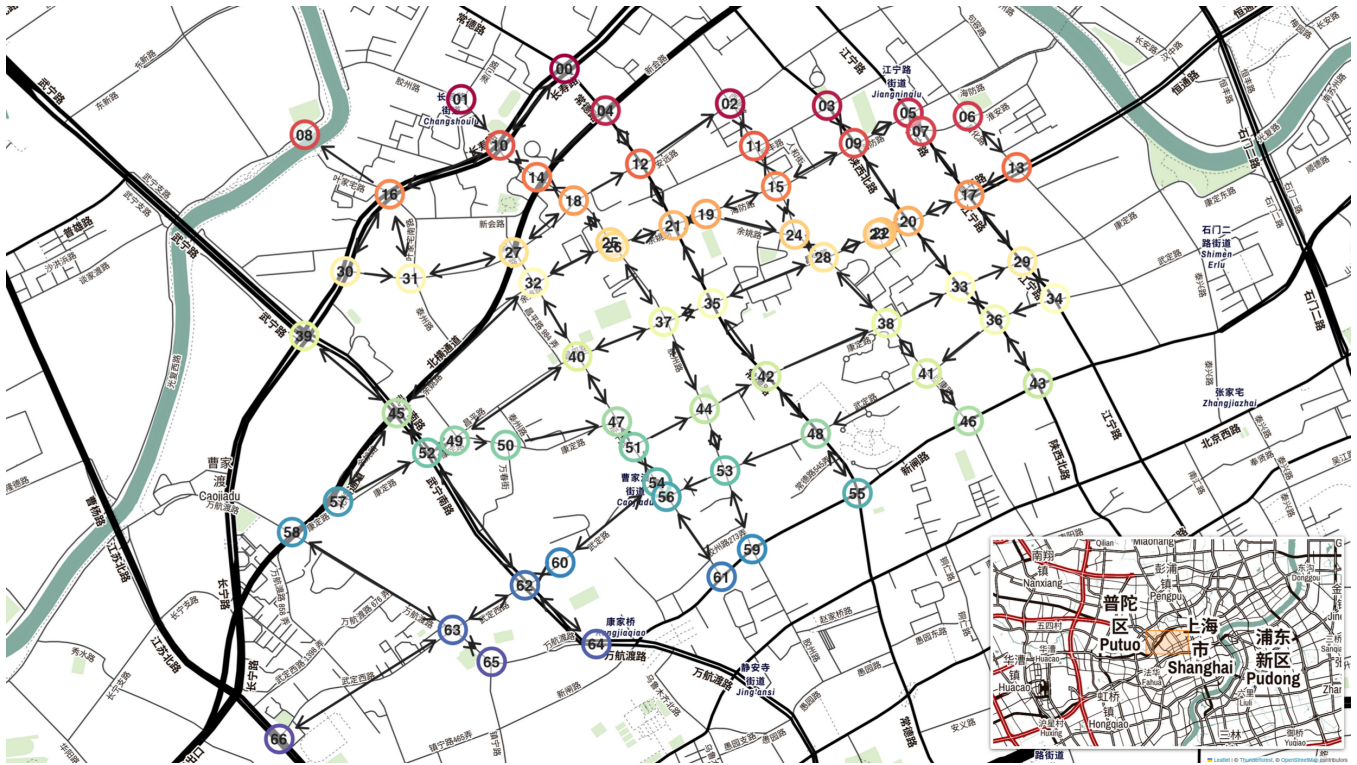


Figure 3.1: The vertices V of the dataset in Shanghai, coloured by their integer representation in order to alleviate finding particular vertices. For an interactive map visit the project's GitHub Pages on finnharbeke.github.io.

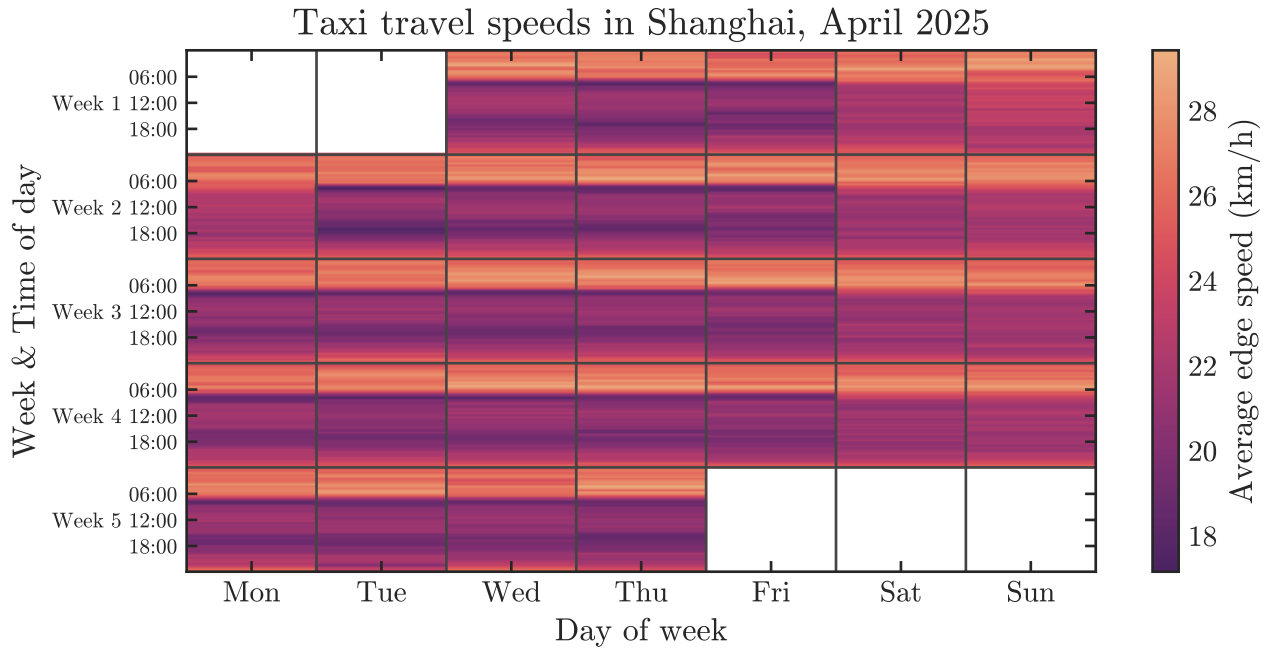


Figure 3.2: The graph-wide mean travel speed for every recorded 10-minute interval in the original dataset used by [Wan+18a]. The intervals span between April 1st and April 30th 2015. They are arranged in a calendar-like view in order to highlight patterns such as rush hours and the less congested weekends.

3.2 The Shanghai Instance

Problem Instance 1 (Shanghai Taxi). Let $G_s = (V_s, E_s)$ be the graph from the Shanghai dataset, where V_s are the intersections and E_s the road segments. Let C_s be the N_s edge costs reported over 10-minute intervals during April 2015 and $\mathcal{D}_s$ the uniform distribution over the set C_s . Then $(G_s, C_s, \mathcal{D}_s)$ is an instance of problem 1.

Instance 1 is a very useful problem instance to start studying the **Stochastic Shortest Path Problem** in an exploratory manner. See table 3.1 for some statistics. With 67 vertices, G_s is by no means a big graph, but it is large enough to possess distinct neighbourhoods and different road policies, such as both one-way and two-way streets. The relatively little size allows for exploring algorithms with exponential running time, that can serve as a reference, see section 5. This is also by virtue of its sparseness, with an average vertex out-degree of less than 2.5. On the other hand there are various sides to the dataset that make clear it will not be able to provide us with the full picture of problem 1 instances on real-life data. Since the graph is a somewhat arbitrarily cut off sub-graph, there are missing edges between some of the outer vertices that would have likely provided the actual shortest paths, see vertices 43, 46, 55, ... along the most south-east street in section 3. Then there seem to be two pairs of vertices, 22, 23 and 25, 26, that are on the same intersection, but don't share any edge. This could be entirely truthful due to some obstruction on the road, but it is likely to be an error in the data. But most importantly the average travel time between any two vertices is typically less than 5, and never more than 10 minutes (fig. 4.1). In 2019, the average commute time in Shanghai was 42 minutes². While this might be longer than the average taxi trip, the taxi trips are likely to also take longer than 3-8 minutes. Thus, we argue that the time savings gained from analysing a graph of this size would probably only affect the last mile of many actual taxi trips. While the last mile problem³ is a well-known problem it typically refers to logistics or public transport networks, rather than taxi or car trips.

Property	Value
Vertices (n)	67
Directed edges (m)	156
Edge cost unit	Travel time (minutes)
Interval duration	10 minutes
Dataset time span	1.-30. of April 2015
Total recorded intervals (N)	4320

Table 3.1: Summary of the Shanghai road network dataset.

²Source: shanghai.gov.cn

³See wikipedia.org

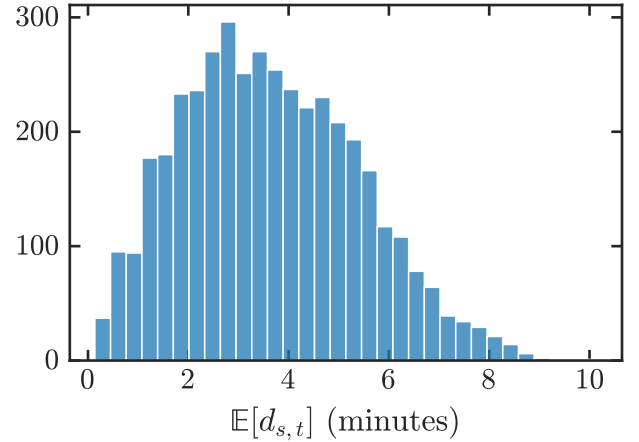


Figure 4.1: Histogram of the expected distance between all (s, t) pairs, where t is reachable s in instance 1.

4 Studying Shortest Paths

In **Stochastic Shortest Path Problem** we are given a query (s, t) , the source and target vertices, as well as the realised cost function $c \sim \mathcal{D}$. The difficulty of returning a short path from s to t varies greatly upon the vertex pair and $\mathcal{D}$. Possibly, t is reachable from s only with a singular path, or there is one path P^* , such that for any $c \in \mathcal{C}$ the cost $f_c(P^*)$ is cheapest possible cost of reaching t from s . In such cases, if we find that path we can be sure we always return it no matter the realisation of c , and the answers are even optimal.

On the other hand, we can easily construct arbitrarily hard graphs and queries. For some pair (s, t) in V , there can be $O(n!)$ many paths that connect them, and for each such P we could assign exactly one cost function in $\mathcal{C}$ where any other path P' costs arbitrarily much more; $f_c(P') > z \cdot f_c(P)$ for any $z \in \mathbb{R}$. For such a difficult setting the portfolio method will be unusable, the probability that a cheap path is in the portfolio would be $O(1/n!)$. In such settings it is prudent to pay in query time and use algorithm 1. Real-world traffic networks are not very likely to behave this way, and that is the point we want to make use of.

Hence, we study the (s, t) -pairs in instance 1 in order to better grasp the structure of their graphs and cost distributions. For that purpose let us find patterns of the distance random variables $d_{s,t}$ for all s, t . ?? shows the expected distance over the vertex pairs in instance 1.

We see there is a wide range of average distances. This is intuitive, shortest path costs should vary greatly for vertices sharing their neighbourhood in comparison to vertices on opposite ends of the entire graph. With similar intuition, we could claim that **Stochastic Shortest Path Problem** decreases in difficulty for (s, t) that are adjacent in the graph. For “behaved” distributions $\mathcal{D}$ there might not be any scenarios, where the direct path using the edge (s, t) is not the most

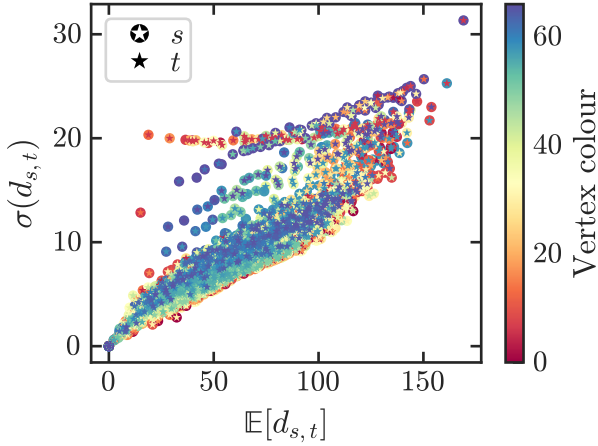


Figure 4.2: The expected distance against the standard deviation of $d_{s,t}$ over all (s,t) pairs in instance 1. Each point corresponding to an (s,t) pair is two-coloured matching the colouring in section 3 in order to more easily find the corresponding pairs there. For a larger, interactive plot visit [this project’s GitHub Pages](#).

sensible answer.

In order to find some first (s,t) -queries that should prove interesting or difficult, we use scatter plots in the following discussion of (s,t) pairs. There, each marker is two-coloured, indicating the values of s and t respectively. Let us first plot the already shown $\mathbb{E}[d_{s,t}]$ against the standard deviation of those same random variables, $\sigma(d_{s,t})$.

Clearly, the expected distance and the standard deviation are very correlated and most points in the plot are found in one big cloud. Interestingly, one can find these strings of points with similar colours that form lines. Upon further inspection we notice that they form pairs of vertices with shared commonly used subpaths. Noteworthy are the (6,66) vertex pair, that is the pair with the furthest average travel distance as well as the highest absolute standard deviation. As well as (6,13), for which the ratio of standard deviation over expected value is the highest. Note (6,13) is an edge in instance 1, as well as the only path from 6 to 13, and it influences the entirety of the red band that is dotted almost horizontally along the grid line of a standard deviation of 20, which are all pairs whose source is 6.

This observation plays into the discussion of the difficulty of a source-target pair very crucially. (6,13) is not a difficult pair, there is only one single path to reach 13 from 6. The high variance of a single edge in the graph does not imply that many different paths are the shortest in distinct events from the distribution. Instead, let us consider a new measure of uncertainty given (s,t) .

The shortest path’s *entropy* is

$$H(P_{s,t}^*) = - \sum_{P \in \mathcal{P}_{s,t}} \Pr(P_{s,t}^* = P) \log \Pr(P_{s,t}^* = P),$$

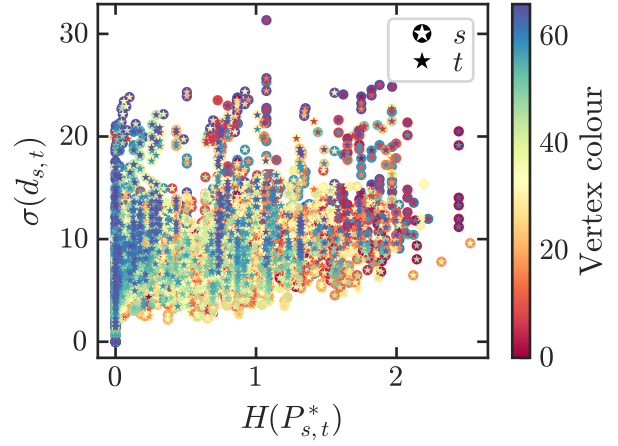


Figure 4.3: The entropy of the shortest path against the standard deviation of $d_{s,t}$ over all (s,t) pairs in instance 1. Each point corresponding to an (s,t) pair is two-coloured matching the colouring in section 3 in order to more easily find the corresponding pairs there. For a larger, interactive plot visit [this project’s GitHub Pages](#).

with the convention that $0 \log 0 = 0$. The entropy measures the uncertainty in which path will be the shortest.

Think of $P_{s,t}^*$ as a categorical random variable taking values like “along the lake” or “the highway”, indicating which path is best as of now. When it is clear that the path is the same for all cost functions $c \in \mathcal{C}$, then **Stochastic Shortest Path Problem** is trivial, a portfolio of size 1 is optimal and we always return that path. But when it takes many different values in many different events, that is when things become more difficult. The entropy of the shortest path should capture a notion that is related to the difficulty of the query (s,t) . Thus, we plot $H(P_{s,t}^*)$ for all s,t in fig. 4.3, in comparison the already discussed $\sigma(d_{s,t})$.

We’re quick to see how different of a picture the shortest path entropy paints in comparison to the standard deviation of the distance. The perfectly vertical strings of points are pairs that share subpaths, and because they have the same entropy they must be using the same subpaths in the exact same events. It is only the edges that are appended to the ends of those subpaths for some pairs that change the standard deviation of the distance considerably.

In essence the entropy describes the uncertainty of what path will be the shortest. With less entropy, it thus is more likely that the shortest path is one that we expect.

5 Portfolios

We propose to solve problem 1 with the framework given in algorithm 2. Given $O(kn^3)$ space, how can we compute portfolios of k paths in polynomial time, such that for any

graph weight realization and query (s, t) , the shortest path in the respective portfolio is close to the optimal shortest path from s to t . In this section we propose three different algorithms solving problem 3.

5.1 Algorithms

If you consider the maximization problem of the portfolio optimization problem, then the value function is a monotone submodular function, as pointed out by [Dry+24] and initially shown by [KR18]. Thus, the widespread *Greedy* algorithm provides a $(1 - 1/e)$ -factor approximation. While there do not exist the same proven guarantees, we can use the same approach to greedily build a portfolio. It builds the solution set iteratively, starting with the path that is the cheapest in expectation and then adding one path at a time to the set, such that at every step the cost is minimised over all candidates. We describe the procedure in algorithm 3.

Algorithm 3 Greedy Algorithm for *implicit* (s, t) -Portfolio.

```

1: procedure GREEDY( $G, \mathcal{C}, \mathcal{D}, s, t$ )
2:    $\mathcal{K}_{s,t} \leftarrow \emptyset$ 
3:    $\mathcal{P}_{s,t} \leftarrow$  the set of all paths from  $s$  to  $t$ 
4:   for  $i \leftarrow 1 \dots k$  do
5:      $P^\dagger \leftarrow \text{none}$ 
6:      $\text{best} \leftarrow \infty$ 
7:     for  $P \in \mathcal{P}_{s,t}$  do
8:       if  $\text{cost}(\mathcal{K}_{s,t} \cup \{P\}) < \text{best}$  then
9:          $P^\dagger \leftarrow P$ 
10:     $\mathcal{K}_{s,t} \leftarrow \mathcal{K}_{s,t} \cup \{P^\dagger\}$ 
11: return  $\mathcal{K}_{s,t}^k$ 

```

Greedy iterates over the entire set $\mathcal{P}_{s,t}$ in each round. Since $\mathcal{P}_{s,t}$ can contain exponentially many paths, its worst-case running time is exponential in n . Even for the modest $n = 67$ of instance 1, enumerating all s - t paths is computationally costly. Still, for any algorithm that builds $\mathcal{K}_{s,t}$ element by element, Greedy is the one that takes the best choice at every step.

An alternative approach is to rank paths by how often they are the shortest. For each path $P \in \mathcal{P}_{s,t}$, define its *frequency* as

$$\Pr(P_{s,t}^* = P) = \Pr_{c \sim \mathcal{D}} \left[P = \arg \min_{P' \in \mathcal{P}_{s,t}} f_c(P') \right].$$

The *most frequent shortest paths* (MFSP) algorithm simply selects the k paths with the highest frequency:

Unlike Greedy, MFSP never enumerates $\mathcal{P}_{s,t}$ explicitly. Its runtime is dominated by N Dijkstra calls, each taking $O(m \log n)$ time, yielding $T^0(\text{MFSP}) = O(Nm \log n)$ per pair (s, t) . The query phase is unchanged: given a realised c , evaluate the k portfolio paths under f_c and return the cheapest.

Both algorithms 3 and 4 rely on repeated shortest-path computations under the full distribution $\mathcal{D}$. A lighter alternative

Algorithm 4 Most Frequent Shortest Paths (MFSP) for *implicit* (s, t) -Portfolio.

```

1: procedure MFSP( $G, \mathcal{C}, \mathcal{D}, k, s, t$ )
2:    $\text{freq}[\cdot] \leftarrow$  empty dictionary
3:   for  $c \in \mathcal{C}$  do
4:      $P \leftarrow \text{DIJKSTRA}(G, s, t, c)$   $\triangleright$  Shortest under  $c$ .
5:      $\text{freq}[P] \leftarrow \text{freq}[P] + 1$ 
6:   return  $k$  paths with largest  $\text{freq}[\cdot]$ 

```

is to use only the *expected* edge costs

$$\bar{c}(e) = \mathbb{E}_{c \sim \mathcal{D}}[c(e)],$$

and obtain k candidate paths by repeatedly running Dijkstra while penalising edges that have already been selected. Concretely, after finding a path P , we multiply the expected cost of every edge in P by a penalty factor $\lambda > 1$, so that subsequent shortest-path computations are discouraged from reusing the same edges and instead explore alternative routes. The procedure is given in algorithm 5.

Algorithm 5 λ -Penalised Dijkstra for *implicit* (s, t) -Portfolio.

```

1: procedure  $\lambda$ -PENALISED DIJKSTRA( $G, \mathcal{C}, \mathcal{D}, k, s, t$ )
2:    $\bar{c} \leftarrow \mathbb{E}_{c \sim \mathcal{D}}[c(\cdot)]$ 
3:    $\mathcal{K}_{s,t} \leftarrow \emptyset$ 
4:    $p[e] \leftarrow 1$  for all  $e \in E$   $\triangleright$  Edge penalties
5:   for  $i \leftarrow 1 \dots k$  do
6:      $P \leftarrow \text{DIJKSTRA}(G, s, t, \bar{c} \cdot p)$ 
7:      $\mathcal{K}_{s,t} \leftarrow \mathcal{K}_{s,t} \cup \{P\}$ 
8:     for  $e \in P$  do
9:        $p[e] \leftarrow \lambda \cdot p[e]$   $\triangleright$  Penalise the used edges.
10:  return  $\mathcal{K}_{s,t}$ 

```

Penalised Dijkstra never consults the full distribution $\mathcal{D}$ beyond computing the per-edge expectation $\bar{c}$. Its preprocessing cost is a single pass over $\mathcal{C}$ to compute means ($O(Nm)$) followed by k Dijkstra calls on the mean-weight graph ($O(km \log n)$). Because it relies solely on expected costs, it ignores the stochastic nature of edge weights entirely: a path that is never the shortest under any realisation but has low expected cost may be selected, while a path that is frequently the shortest under many $c \in \mathcal{C}$ may be missed if its expected cost is high.

5.2 Quantitative Analysis

In section 5.1 we described three different approaches to solve problem 3. With the framework given in algorithm 2, we thus have three full algorithms for problem 1, all of them are parametrised by k while λ -penalised Dijkstra has an additional parameter. Taking instance 1, we can evaluate their performance on that graph and its edge cost distribution. As

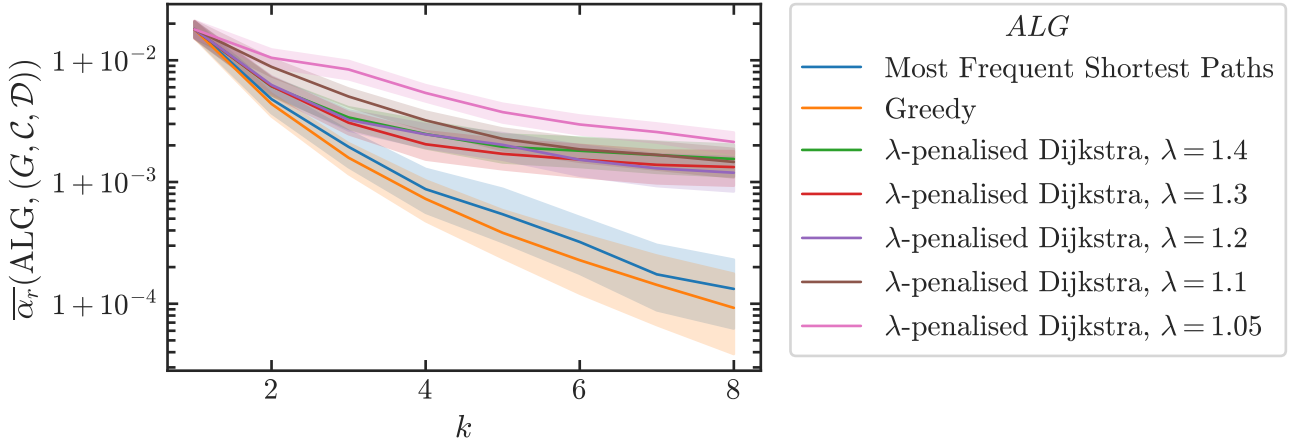


Figure 5.1: The observed mean of the expected approximation factor of the algorithms presented in section 5.1, when used as the PORTFOLIOALG for algorithm 2. The mean is observed over r uniformly sampled (s, t) pairs of the graph in instance 1, the Shanghai Dataset, where $r = 334$ for λ -penalised Dijkstra and $r = 167$ for the other algorithms. The error bands are 95% confidence intervals on the sampled expected approximation factors.

discussed in section 2, for a query (s, t) we are interested in the ratio between the expected cost of the algorithm’s answer and the expected cost of the shortest path, denoted by $\alpha(\text{ALG}, \text{instance 1}; s, t)$. I.e. how much more does the answer cost than optimally possible. Over r uniformly sampled (s, t) queries we then estimate the expected value of that ratio for any query with $\bar{\alpha}_r(\text{ALG}, \text{instance 1})$. In fig. 5.1, those numbers are visualised.

At $k = 1$ all algorithms achieve nearly the same expected approximation factor of roughly 1.018. As k grows, every algorithm improves monotonically as expected, but the rates at which they do differ sharply. Greedy and MFSP drop quickly: their extra cost over optimal is already below 0.5% at $k = 2$ and reaches mere hundredths of a percent by $k = 8$. By contrast, the λ -penalised Dijkstra variants decay more slowly—the best-tuned case ($\lambda = 1.2$) retains an excess of roughly 0.12% at $k = 8$, an order of magnitude worse than Greedy or MFSP. This must be because penalised Dijkstra, only using expected edge costs and reweighing edges, yields paths of a diversity that is different from the diversity that is valuable to portfolios. MFSP performs similarly to Greedy across all k , despite never enumerating the full path set and requiring only a fixed number of Dijkstra runs in preprocessing. This is a positive result, given MFSP is one of the simplest polynomial-time algorithms that can make use of information in the entire distribution.

The central tradeoff we are exploring with the algorithm 2 approach to solving [Stochastic Shortest Path Problem](#) is the one between answer quality and query runtime. algorithm 1 is the one extreme, that does all the work at query time. It has a query runtime $T^q(\text{ALG})$ of $O(m \log n)$ and grants perfect answer quality. The query runtime of algorithm 2 is $O(kn)$, for evaluating the costs of each of the k paths in $\mathcal{K}_{s,t}$. With a

higher k runtime increases linearly, but the expected approximation factor decreases, we can even hit perfect scores like Dijkstra. From a sample of different (s, t) query pairs, we visualise the tradeoff in fig. 5.2. For the sake of visual clarity, we omit the results of λ -penalised Dijkstra. We see that for all pairs, the query runtimes are less than that of Dijkstra for portfolios with $k = 3$ or less. Dijkstra’s queries take about the time of evaluating portfolios with 4-6 paths. For bigger graphs, Dijkstra should only get slower in comparison if there are more than a linear amount of edges.

Before we proceed to qualitatively evaluate the algorithms in the next section, we emphasise some of the core differences of algorithms 3 to 5 in table 5.1.

ALG	Uses $\mathcal{D}$?	Full $\mathcal{P}_{s,t}$?	$T^0(\text{ALG})$ per pair
Greedy	Fully	Yes	$\Omega(n!)$
MFSP	Only $P_{s,t}^*$	No	$O(Nm \log n)$
Pen. Dijkstra	No	No	$O(Nm + km \log n)$

Table 5.1: Properties of the three portfolio algorithms from section 5.1.

While we have to constrain ourselves to the Shanghai Dataset in this report, some first evaluations have been made on the [METR-LA](#) dataset, where similar behaviour can be observed while the overall difficulty is higher. The single-path portfolios start off with a clearly higher cost increase, and for $k = 8$ MFSP achieves expected approximation factors of around 1.002, whereas for Shanghai it is around 1.0002. For a short insight see fig. A.1.

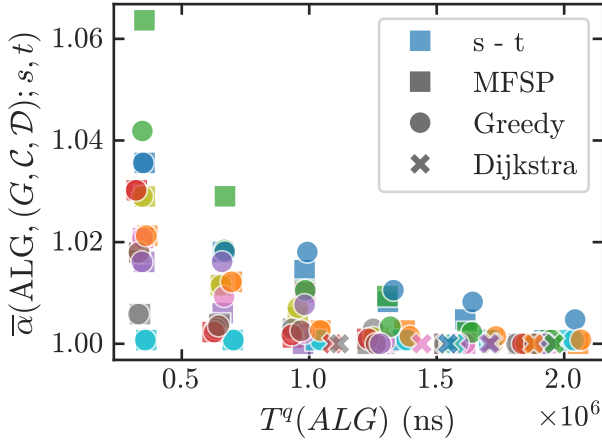


Figure 5.2: The expected approximation factor of 10 (s, t) -queries compared to query runtime measured in nanoseconds. Each (s, t) query is given a colour, and besides the results of Dijkstra, portfolios from Greedy and MFSP are used with $1 \leq k \leq 6$. The queries are the first 10 of uniformly sampled, reachable (s, t) pairs, where Greedy’s portfolio for $k = 3$ did not already obtain perfect answers. For reference, this applied to only 18 of 50 samples.

5.3 Qualitative Analysis

Let us take a look at an example query in instance 1 and how the different algorithms preprocessed the graph to compute a suiting portfolio. A particular portfolio induces a partition on $\mathcal{C}$, we can group its cost functions by which ones assign the first path the cheapest cost, which ones assign the second path the cheapest cost and so on. Let us denote that partition for a given portfolio $\mathcal{K}$ by

$$Q(\mathcal{K}) = \{ \{c \in \mathcal{C} \mid \forall P' \in \mathcal{K} \ c(P) \leq c(P')\} \mid P \in \mathcal{K} \}.$$

Every set in $Q(\mathcal{K})$ is thus also an event of the probability space of the distribution $\mathcal{D}$. It is the event that “ P is the cheapest of all paths in $\mathcal{K}$ ”. We denote its probability by

$$\Pr[P \text{ wins } \mathcal{K}] = \Pr[\{c \in \mathcal{C} \mid \forall P' \in \mathcal{K} \ c(P) \leq c(P')\}].$$

If $\Pr[P \text{ wins } \mathcal{K}]$ is evenly distributed across all $P \in \mathcal{K}$, then each path carries its weight and the portfolio covers the distribution broadly. But even a path that is unlikely to win overall can be valuable: when it does win, it may bring a significant decrease in cost compared to the other candidates, justifying its place in the portfolio.

For a single set in a partition $Q(\mathcal{K})$, where some path P is the one that wins, we can observe the typical “traffic situation” for that event. Let us define the relative mean cost of an edge $e \in E$ in that set, as a ratio with the overall mean cost of e :

$$\frac{\bar{c}_{P, \mathcal{K}}(e)}{\bar{c}(e)} = \frac{\mathbb{E}_{c \sim \mathcal{D}}[c(e) \mid P \text{ wins } \mathcal{K}]}{\mathbb{E}_{c \sim \mathcal{D}}[c(e)]}$$

For the pair $(17, 49)$ in instance 1, we see the paths chosen by Greedy and MFSP, as well as the respective $\Pr[P \text{ wins } \mathcal{K}]$ and the $\frac{\bar{c}_{P, \mathcal{K}}(e)}{\bar{c}(e)}$ for all edges on the paths visualised in fig. 6.1. The figure contains a lot of information, but we think it is all necessary information to be able to get some insight on the given portfolios and their differences. The chosen pair is a very difficult query, in the sense that the expected approximation factors above 1.01 are clearly outside of the 95% confidence intervals for $k = 3$ in fig. 5.1. This must be due to many different paths with similar distances between the source and target. We see that the two vertices are sort of on two ends of a manhattan-like grid. The edge that Greedy has over MFSP is well emphasised by the respective choices of the third path in the portfolio. Greedy is able to find the path that shines only rather rare scenarios, but when it does it shines brightly and can bring the biggest decrease in expected cost of the portfolio. Such a path will not be found by MFSP until a larger k is being used. For this query λ -penalised Dijkstra actually finds the same path as the third part of its portfolio, but chooses a second path that is less similar to the red path than the blue one and thus still has a worse expected approximation factor than MFSP’s portfolio. fig. A.2

6 Graph-wide data structure

Up until now, the main framework we used to solve problem 1 has been algorithm 2. Computationally, this offers a lot of potential for saving resources by spending $T^0(\text{ALG}) = O(n^2) \cdot T(\text{PORTFOLIOALG})$ time just once, and then profiting off of smaller $T^q(\text{ALG})$. Regarding memory, this can still be costly. The space needed to store the portfolios computed in the preprocessing is $O(kn^3)$, since we store $O(n^2)$ many portfolio of k paths with a length of up to n vertices. While the average path length might be much less than n , even paying for space whose size is quadratic in n can be too expensive.

In this section we thus discuss an approach that uses significantly less space. Consider any portfolio $\mathcal{K}$ for an instance of problem 3. As discussed in section 5.3, $\mathcal{K}$ implies a partition $Q(\mathcal{K})$ on the cost functions.

Similarly we can deduce a portfolio from some partition Q .

$$K^*(Q) = \{ \arg \min \{ \mathbb{E}[c(e) \mid c \in R] \} \mid R \in Q \}$$

Note that $K^*(Q)$ is the best portfolio out of all portfolios that induce the identical partition. Thus $\text{cost}(\mathcal{K}^*(Q(\mathcal{K}))) \leq \text{cost}(\mathcal{K})$.

The idea of our algorithm is that we sample (s, t) queries uniformly and compute the k -partitions induced by the k -portfolios computed for each pair. This gives us a collection

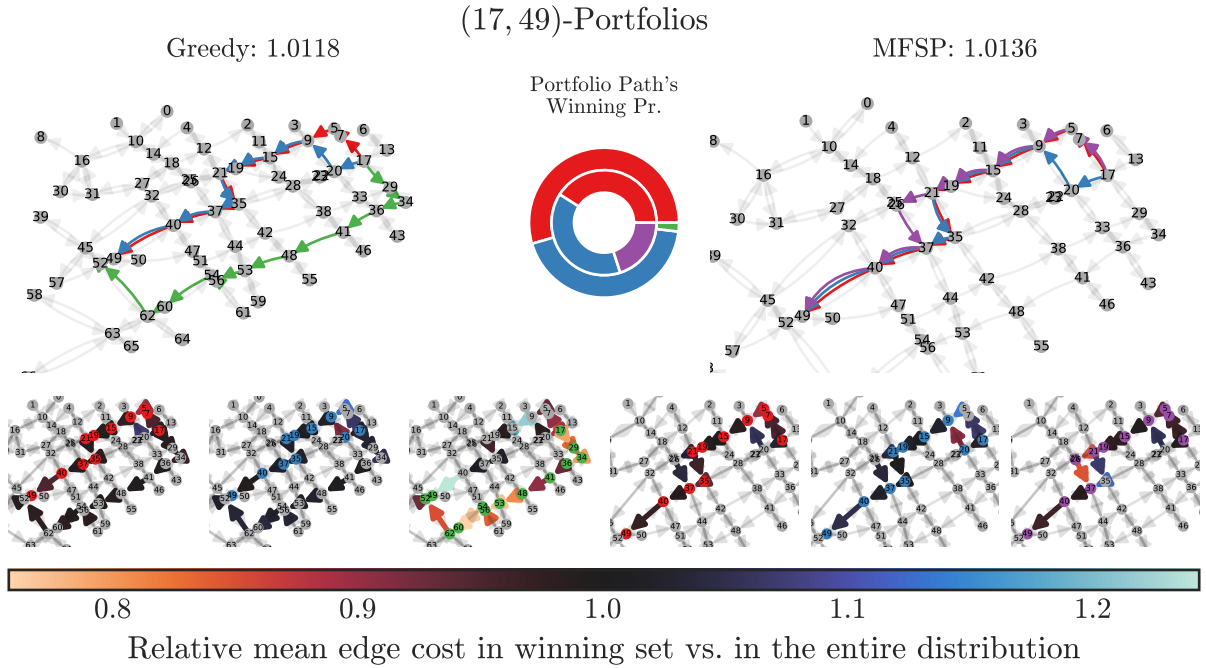


Figure 5.3: Comparison of the two portfolios with $k = 3$ computed by algorithms 3 and 4 for the pair $(17, 49)$. The chosen paths are displayed in the top-left and top-right subplots respectively with their expected approximation factors in the title. The pie chart visualises $\Pr[P \text{ wins } \mathcal{K}]$ for all P of Greedy's portfolio in the outer ring, and those of MFSP's portfolio in the inner ring respectively. Greedy's third path (green) has low probability of winning, but is the best match for the red and blue paths by Greedy's definition. MFSP finds the same two first paths, but then continues with another path (purple) along similar vertices, because it is another dominant shortest path, but not an as much cheaper alternative as the green one is. The bottom row of subplots visualise $\frac{\bar{c}_{P,\mathcal{K}}(e)}{\bar{c}(e)}$ for the three paths of the left portfolio and for the three paths of the right portfolio.

of partitions Q . These we aggregate to a single partition $\bar{Q}$ that is representative of Q .

Given $\bar{Q}$ we can separate G into k non-stochastic graphs, each having the mean edge costs of one of the sets in $\bar{Q}$. We can now proceed with using any known algorithm that can preprocess fixed graphs and answer shortest-path queries fast. A field of research that aims to solve such problems tries to introduce hierarchies to the graph nodes or edges, only precomputing paths between vertices in the same level.

For this project, there was not enough time left to properly investigate the results such an algorithm would yield. In order for it to work there has to exist some partition of the cost functions, that is able to represent the different traffic scenarios throughout the entire graph. If for every (s, t) query we find entirely new partitions, then there might not be any way to aggregate them.

Assuming a representative partition of the cost functions can be found it would be interesting to visualise that partition on a calendar-like chart as in section 3.1. Is there more to it than just finding the rushhours, night times and weekends?

We ran some first experiments trying to find some partition $\bar{Q}$. For an increasing number of $q = |Q|$ we ran 20 experiments each, ending up with a $\bar{Q}$ for each. We aggregated the sets of partitions using the k-medoids algorithm and the hamming distance between two partitions. If there was some representative partition emerging, then those $\bar{Q}$ should be similar. We are plotting the hamming distances between these in ?? . As one can see the obtained partitions from re-running experiments differ quite strongly, hinting towards the difficulty of finding meaningful graph-wide cost function partitions. For reference, the hamming distance between uniformly random partition has an expected value of 2880, which is two thirds of the number of cost functions in instance 1. There is a clear downwards trend as q increases, so one would have to definitely study what happens for clearly larger amounts of (s, t) queries. Maybe more representative data can be found.

7 Conclusion

In this project we studied the stochastic shortest path problem through the lens of portfolio optimization. We framed the problem as selecting, for each source–target pair, a small set of candidate paths in a preprocessing phase, such that at query time we can return the cheapest among them under the realised edge costs. We implemented three portfolio algorithms—a greedy selection over the full path set, a frequency-based selection of the paths that are shortest the most often, and a penalty-based scheme that iteratively penalises edges of expected-cost shortest paths—and evaluated them on a real-world traffic dataset from Shanghai.

Our main finding is that on this instance, portfolios of moderate size achieve near-optimal expected costs. The greedy and frequency-based algorithms both reach expected cost in-

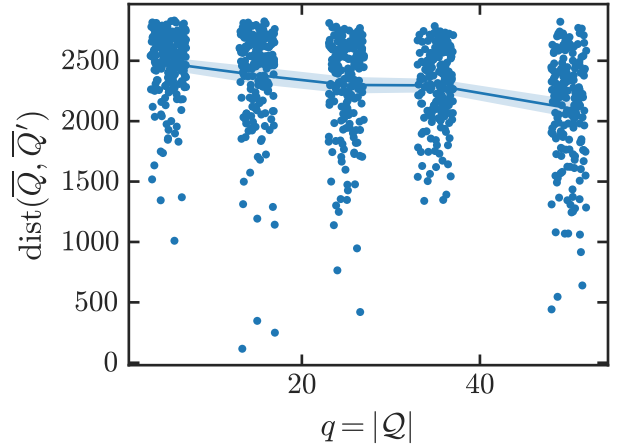


Figure 6.1: Pairwise distances between 20 “representative” partitions obtained from a collection of q partitions for different values of q .

creases within hundredths of a percent compared to the optimal with $k \approx 8$ paths, and the frequency-based approach does so without enumerating the exponentially many candidate paths. This suggests that for traffic networks with similar characteristics, the portfolio model can provide a meaningful reduction in query-time computation without a substantial loss in solution quality. The penalty-based scheme, while cheaper to precompute, converges more slowly, indicating that information beyond the mean edge cost matters when building diverse portfolios.

These results come with several caveats. The Shanghai dataset is small ($n = 67$, $m = 156$) and covers a limited geographic area where trips are short. The graph is a subgraph of a larger network and may contain artifacts from the way it was extracted. It is not clear how well the findings transfer to larger graphs, longer trips, or different types of traffic data. Our evaluation is also limited to one instance and one distribution; the difficulty of the stochastic shortest path problem depends heavily on the graph structure and the nature of the cost uncertainty, as we discussed in section 4.

7.1 Future work

A natural next step is to evaluate the portfolio approach on additional datasets. The Shanghai dataset used here is small; datasets for Shenzhen (156 vertices) and Los Angeles (207 vertices) are available from the other sources and would provide a useful check on whether the trends we observed persist in larger graphs with different connectivity patterns. But it is difficult to obtain continuous time series and for example the data from Shenzhen has time intervals with no recordings. There is a point to be made towards defining the cost distribution differently, instead of a uniformly chosen timepoint throughout the month, one might want to sample uniformly

from all observed car speeds, giving much more weight to rush hours and eliminating the problem of time intervals without traffic. Datasets with longer time spans or coarser time aggregation might also change the nature of the uncertainty and thus the value of portfolios.

On the theoretical side, it would be interesting to study if one can identify conditions on the distribution $\mathcal{D}$ under which a constant-factor approximation guarantee can be proved for one of the proposed algorithms. For example, if the edge cost distributions are independent and have bounded variance, one might be able to bound how far the expected-cost shortest path can be from the true shortest path in any realisation. Such results would complement the empirical evaluation we presented here.

The memory cost of storing per-pair portfolios grows as $O(kn^3)$, which becomes prohibitive for graphs with thousands of vertices. The unfinished work in section 6 points toward data structures that store a single graph-wide set of paths and derive per-pair portfolios from it on the fly. There are also connections to learning-based approaches: one could train a model to predict, from some features of the query and the current traffic conditions, which of a small set of global routes is likely to be cheapest. Such methods could trade some theoretical guarantees for better scalability.

8 Acknowledgements

I want to thank Miltiadis Stouras for the supervision and the fun discussions throughout the semester.

References

- [Dry+24] Marina Drygala, Silvio Lattanzi, Andreas Maggiori, Miltiadis Stouras, Ola Svensson, and Sergei Vassilvitskii. *Data-Driven Solution Portfolios*. 2024. arXiv: 2412.00717 [cs.DS]. URL: <https://arxiv.org/abs/2412.00717>.
- [KR18] Jon Kleinberg and Maithra Raghu. *Team Performance with Test Scores*. 2018. arXiv: 1506.00147 [cs.DS]. URL: <https://arxiv.org/abs/1506.00147>.
- [Wan+18a] Xiaoyu Wang, Cailian Chen, Yang Min, Jianping He, Bo Yang, and Yang Zhang. *Efficient Metropolitan Traffic Prediction Based on Graph Recurrent Neural Network*. 2018. arXiv: 1811.00740 [cs.AI]. URL: <https://arxiv.org/abs/1811.00740>.
- [Wan+18b] Xiaoyu Wang, Cailian Chen, Yang Min, Jianping He, and Yang Zhang. “Vehicular Transportation System Enabling Traffic Monitoring: A Heterogeneous Data Fusion Method”. In: *2018 10th International Conference on Wireless Communications and Signal Processing (WCSP)*. 2018, pp. 1–7. DOI: [10.1109/WCSP.2018.8555669](https://doi.org/10.1109/WCSP.2018.8555669).

A Appendix

The code for the entire project can be found on [Github.com](https://github.com).

Then we share some LLM sessions from [Pi-Agent](#).

- The [first](#) was used for some paragraphs in the main section, along the way.
- The [second](#) was used to draft the introduction and the conclusion
- And the [last](#) helped with adjusting intermediate plots to the final figs. [A.2](#) and [6.1](#).

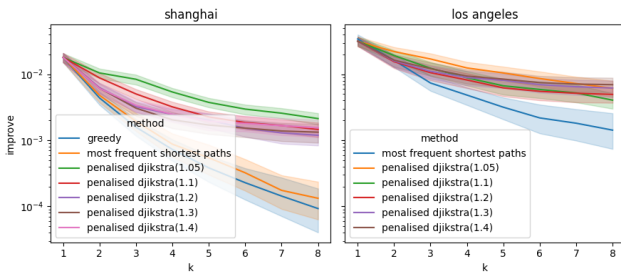


Figure A.1: We also ran experiments on the [METR-LA](#) dataset. This figure is an analogue to [fig. 5.1](#)

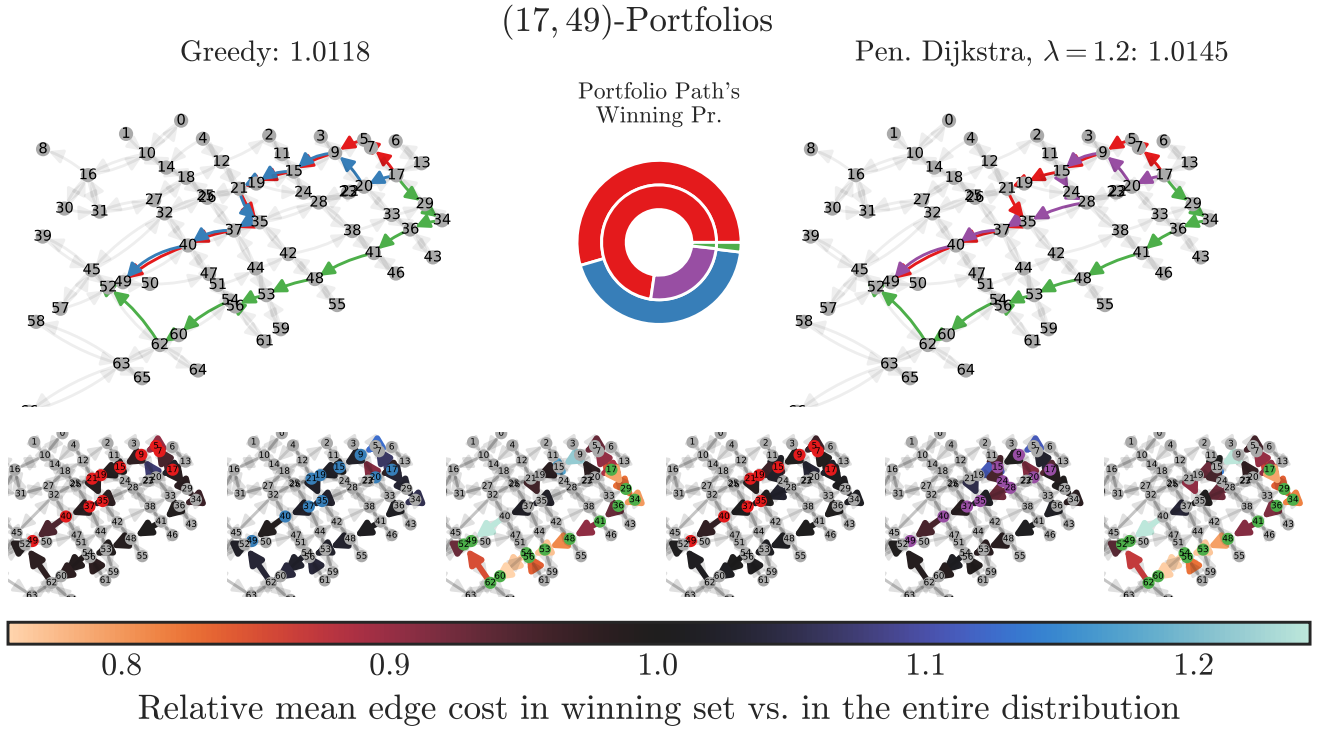


Figure A.2: Comparison of the two portfolios with $k = 3$ computed by algorithms 3 and 5 for the pair (17, 49). The chosen paths are displayed in the top-left and top-right subplots respectively with their expected approximation factors in the title. The pie chart visualises $\Pr[P \text{ wins } \mathcal{K}]$ for all P of Greedy's portfolio in the outer ring, and those of Penalised Dijkstra's portfolio in the inner ring respectively. The bottom row of subplots visualise $\frac{\bar{c}_{P, \mathcal{K}}(e)}{\bar{c}(e)}$ for the three paths of the left portfolio and for the three paths of the right portfolio.